

Matteo Palmonari

matteo.palmonari@disco.unimib.it

Modeling Knowledge Graphs: *RDFS and OWL Hands-on Session* *Exercises **Only***



dat·AI

data * AI
AI * data

RDFS and OWL



Hands-on Session

- *Ontology-based Data Modeling
- *RDFS/OWL Axioms and Reasoning
- *Modeling patterns (subclass-of / part-of)

Structure of the Hands-on Session

- Video introduction to Protégé
 - How to edit ontologies with Protégé (3 videos)
- Task specification without solutions
 - Levels 1-2-3 (to do); Level 4 (not mandatory)
- Task specification + **solutions and comments**
 - Solutions are provided as .owl files that you can upload and check in Protégé + comments in the slides
 - Includes important “take-home messages” at the end

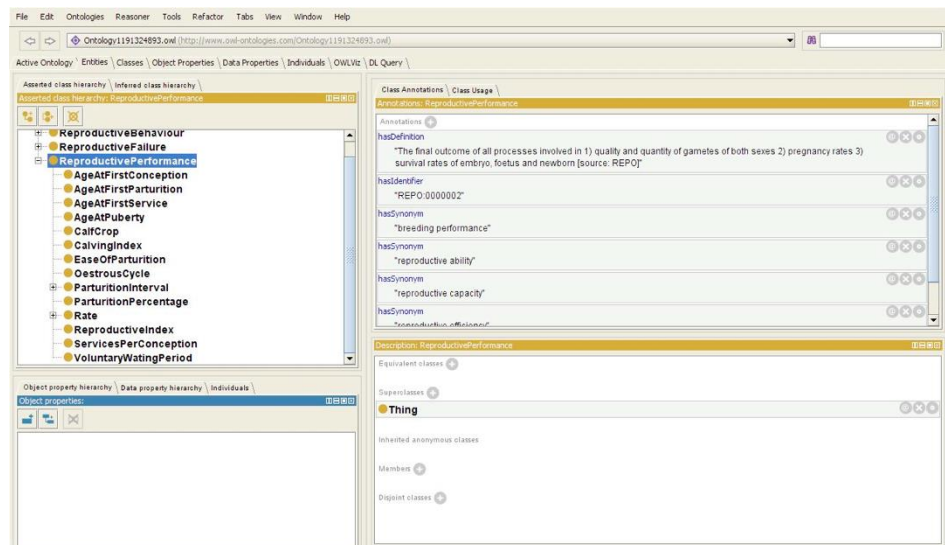
This presentation

Ontology Editing

State-of-the-art open source editor:
Protégé

- Graphical support for design and editing (for TBox and ABox)
 - Editing and representation with Description Logic syntax and automatic generation of OWL and RDF code
 - Basic checks
 - Visualization Tools

Download Protégé at:
<https://protege.stanford.edu/>



Integration with state of the art
reasoners, rule based systems and
query systems

Check the short video
tutorials about Protégé
before you start

Start to try it by opening an existing ontology, e.g., the Pizza ontology.

→ “Open from URL” using ”<http://www.cs.ox.ac.uk/isg/ontologies/UID/00793.owl>”

Also check: <https://protege.stanford.edu/support.php>

Ontology Modeling: High-level Objectives

- Build an ontology with Protégé, which represents geographical entities, organizations and their connections using axioms in RDFS/OWL
 - test different modeling tasks in Protégé
- Populate the ontology with few instances
 - remember that the goal of terminological axioms (Tbox) is to model data (Abox) not to define an abstract model
- Iteratively use the reasoner to check inferences computed from the axioms
 - Get confident with reasoners and understand the consequences of the introduced axioms

Ontology Modeling: Recommendations

- You need to fulfill the specifications (specs), which are grouped into three different levels
 - the ontology should consider all the concepts / properties / assertions specified, but
 - you can extend the ontology as you want, e.g., introduce new concepts / properties to better organize the ontology
- Use always the same naming strategy:
 - CamelCaseClass; camelCaseProperty
 - Underscore_Class; underscore_Property
- Read all the specs for each level before starting to edit the ontology, but it could be helpful to check solutions after each level
- **Remark:** I won't provide an image (e.g., a graph) as input, because this would be part of the solution and it could be in fact be an intermediate step of your exercise

Ontology Modeling: Level 1 – Specs 1

- Introduce the following classes:
 - Continent, Country, Region, City
 - Organization, Company, NonProfitOrganization [=NGO]
- For Country, define subclasses on a geographical basis (eg EuropeanCountry),
 - to support grouping of countries based on the continent in which they appear
- Define entities for each of the classes (Continent, Country, Region, City) and (Company, NoProfitOrganization), as suggested below:
 - Continents (all); Countries (Italy, Germany, France, Netherlands); Region (Lombardy, Lazio, Piedmont); Cities (Milan, Roma, Berlin, Bonn, Paris); Company (FIAT, SAP), NGO (Emergency, MediciSenzaFrontiere)
 - **Remark:** to fulfill specs 3 about reasoning, where you will be asked to infer types for some of the entities, it is a good idea to specify the most specific types for some of these entities and underspecify types for some others; as a suggestion, keep Italian entities to test reasoning, so underspecify their types when inserting them (e.g., make them instances of owl:Thing).

Ontology Modeling: Level 1 – Specs 2

- Introduce properties to model the relationships between individuals of their respective classes (in the Tbox and in the Abox):
 - A property to link the **contained** geographical entities to those that contain them
 - A specific property to link cities to the countries they are **capital of**
 - A property to specify the **population** of countries (based on Wikipedia)
 - A property “hasHeadquartersIn” to specify the country where an organization has its headquarters
 - A property “hasOfficesIn” to specify where an organization has offices, including headquarters, but not limited to headquarters
- Introduce at least two Abox assertions for each property
 - For region, I suggest using Italian regions: Lombardy, Lazio
- Introduce at least two annotations using `rdfs:label`
 - Suggestion: use MSF for Medici Senza Frontiere and add “Medici Senza Frontiere” as a label; use NGO for NonProfitOrganization and add the full name as `rdfs:label`

Ontology Modeling: Level 1 – Specs 3

- Specify the meaning of the properties using only RDFS constructs (domain/range restrictions)
- Use a reasoner (e.g., Hermit) and calculate the inferences making sure that:
 - The Abox is defined in such a way that you can infer at least one-two classes for the instances, which were not explicitly defined (e.g., infer the type City for some city)
 - Suggestion (see also Specs 1): test type inference on Italian entities;

Ontology Modeling: Level 2

- Specify disjointness between classes
- Specify new properties (one or more, generic, e.g., contains, or type-specific) as inverse of the property used to model part-of relations
 - Suggestions: do not specify domain / range restrictions for this one, see inferred domain / range restrictions using the reasoner; check Abox inferences after reasoning.
- Specify which properties are functional
- Specify which properties are transitive
- **Remarks:**
 - Run the reasoner at each step and check inferences (otherwise it will be hard to debug)
 - Pay attention to the interaction between class disjointness and domain / range restrictions to avoid inconsistencies
 - Pay attention to the interaction between axioms that specify functional and transitive properties, and, also subproperties

Ontology Modeling: Level 3 - Specs

1. Introduce an entity that represents “Rome the Capital”
 1. specify its major (a datatype property is fine if you want to avoid adding also persons to the ontology), and state that this is the same entity as Rome and control that for each of the two entities you infer the same statements.
2. Define the range of **hasHeadquartersIn** as to be either a City or a Country (using the “class expression editor in Protégé”
 - ☐ Under the not-too-realistic - but somehow supported by the information I found in Wikipedia – assumption that headquarters are either specified as the city where they are, e.g., Paris for msf, or as a country, e.g., Netherland for FCA
3. *Define* the following classes (using the Protégé’s “object restriction creator” when adding the equivalent class):
 1. Capital, as the class equivalent to the class of cities that are capitals of at least one Country.
 2. EuropeanCountry, as the class equivalent to *the class of countries that are part of Europe**; to represent *the latter class*, you need to use the following construct (see the DL and Protegé syntax), which we have only quickly mentioned in the lesson:

$\exists P.\{ e \}$ with *e* being an individual

P value *e* with *e* being an individual

1. EuropeanCapital, as as the class equivalent to the class of capitals that are capitals of at least one EuropeanCountry (requires 2 to be specified first)

Ontology Modeling: Level 4 – Specs (for the bravest 😊)

- Try to substitute each of the type specific properties that links cities to regions, regions to countries and countries to continents with one unique non transitive but functional part of property; then specify that for this property, the range must be: regions, if the subjects are cities; countries, if the subjects are regions; continents, if the subjects are countries.
- To do so, use localized domain / range restrictions instead of global (i.e., rdfs) domain and range restrictions, i.e., (slide 42 of the presentation about OWL):

$C \sqsubseteq \forall P.D$

$C \sqsubseteq P \text{ only } D$

Latest Minor Tasks

- Upload the DBpedia ontology in Protégé and check it
 - You will see the solutions adopted to consider the trade-offs discussed in previous slide: much underspecified ontology to avoid inconsistencies
- Export the HTML documentation automatically generated by Protégé from your ontology and check it
 - Tools → Export OWLDoc